

```

    putrightfork ( );
    putleftfork ( );
}

```

- Here philosophers operates asynchronously and concurrently, it is possible for each philosophers to enter into eating mode. The procedure takeleftfork ( ) waits until the specified fork is available and then seizes it. This is not possible because of the following reasons.
- Each philosopher will hold exactly one fork, and no forks will remain available on the table. The philosophers will all deadlock.
- To solve this problem, write some extra code in takenrightfork ( ) procedure. User can specify the philosophers to put down the left fork if that philosophers cannot obtain the right fork.

#### Problem analysis :

- **Shared resource** : Here each fork is shared by two philosophers so we called it is a shared resource.
- **Race condition** : we do not want a philosopher to pick up a fork that has already been picked up by his neighbor. This is a race condition
- **Solution** : we consider each fork as a shared item and protected by a mutex lock. Before eating, each philosopher first lock left fork and then right fork. If philosopher acquires both locks successfully, then philosophers have two locks (two fork) so he can eat a food. After finishes eating, this philosopher releases both fork, and thinks.

END...*✍*

# 4

## Memory Management

### Unit IV

#### 4.1 : Memory Management Concept and Requirements

**Q.1** Explain primary memory requirement. Also list functions of memory management.

**Ans. :** Primary memory requirements

1. **Access time** : It should be as small as possible. This need influences both software and hardware design.
2. **Size** : Size must be as large as possible. It can accommodate many programs into memory.
3. **Cost** : Cost of the memory is less than the total cost of the computer.

#### Memory Management Function

1. Allocate primary memory space to processes.
2. Minimize access time.
3. Determining allocation policy for memory.
4. Deallocation technique and policy.

**Q.2** Define the following :

**Compile time, load time, execution time.**

**Ans. :** • **Compile time** : Source program is translated at compile time to produce a relocatable object module. At compile time, the translator generates code to allocate storage for the variable. This storage address is used for code reference. Target address is unknown at compile time, it cannot be bound at compile time. Example of compile time binding is MS DOS.com programs.

• **Load time** : Compiler generates relocatable code if compile time binding is not performed. The loader modifies the addresses in the load

module at load time to produce the executable image stored in main memory. Final binding is delayed until program load time.

- **Execution time** : Memory address of the program is changed at execution time, then execution time binding is used. Binding is delayed until the run time of the program. Normally all operating system uses execution time binding. Special hardware is used for execution time binding.

### Q.3 What are requirement for memory management ?

ES [SPPU : May-16, End Sem, Marks 8]

Ans. : • Memory management requirements are Relocation, Protection, Sharing, Logical organization and Physical organization.

- **Relocation** : Programmer does not know where the program will be placed in memory when it is executed. While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated). Memory references must be translated in the code to actual physical memory address.

- The operating system need to know the location of :

1. Process control information
2. Execution stack
3. Entry point to begin the execution of a program. 'Processor' must deal with memory references within the program.

- **Protection** : Memory protection is used to avoid interference between programs existing in main memory. The memory protection hardware compares every memory address used by the program with the contents of two registers (base and limit) to ensure that it lies within the allocated memory area. Multiple hardware memories are used to provide a larger address space.

- The simplest method of memory protection is adding two registers to the CPU. This works good for all memory is allocated contiguously. Non-contiguous memory is harder to protect.
- **Sharing** : Allow several processes to access the same portion of memory. Better to allow each process access to the same copy of the program rather than have their own separate copy.

- **Logical organization** : Main memory is organized as a linear one-dimensional address space that consists of sequence of bytes, words. Secondary memory at its physical level is similarly organized. Most of the programs are organized into modules.

- **Physical organization** : Computer memory is organized into levels :

- a) Main memory : Main memory is a volatile memory and it provide fast access at relatively high cost.
- b) Secondary memory : Secondary memory is a non-volatile memory and it is slower and cheaper than main memory

### 4.2 : Memory Partitioning : Fixed Partitioning and Dynamic Partitioning

#### Q.4 Explain fixed partition of memory.

Ans. : Contiguous Memory Allocation with Fixed Partitions

- In a fixed size partitioning of the main memory all partitions are of the same size. The operating system divides main memory into a number of fixed size partition. Each partition holds a single program.
- Fixed sized partitions are relatively simple to implement. Degree of multiprogramming depends on the number of partitions.
- Normally main memory is divided into two partitions :
  1. For resident program
  2. For user processes
- Batch operating system uses the fixed size partition scheme. The operating system keeps the record of memory allocation and deallocation in the form of table. Initially all the memory is available for user processes.
- When the process arrives in the system and needs memory, operating system search for large hole for this process. Hole is one large block of free available memory. If any free hole is found, process is allocated to the free hole of memory as is needed.
- After allocating number of holes for the processes, a set of various size holes is scattered throughout memory at any given time. When a



process arrives and searches for (memory) set of holes. But the holes must be large enough to accommodate the process.

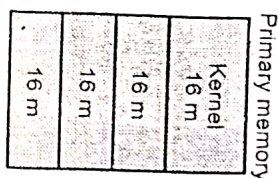
- Fixed sized partitions are simple to implement. Any process whose size is less than or equal to the partition size can be loaded into any available partition.

• Fig. Q.4.1 shows an example of fixed partitioning of a memory.

- There are two difficulties with the use of equal size fixed partitions :

1. A program may be too big to fit into a partition.
2. Memory utilization is extremely inefficient.

• First difficulty is solved by using overlays. Overlays involves moving data and program segments in and out of memory.



#### Advantages and disadvantages of fixed partition size

##### 1) Advantages

1. Simple to implement.
2. Does not require expertise to understand and use such a system.
3. Less overhead.

##### 2) Disadvantages

1. Memory is not fully utilized.
2. Poor utilization of processors.
3. User's process being limited to the size of available main memory.
4. Requires contiguous loading of entire program.

#### Q.5 Explain dynamic partition of memory.

Ans. : Contiguous Memory Allocation with Variable Partitions

- The use of unequal size partitions provides a degree of flexibility to fixed partitioning. In dynamic partitioning, the partitions are of variable length and number.
- In noncontiguous memory allocation, a program is divided into blocks that the system may place in nonadjacent slots in main memory.
- This allocation method do not suffer from internal fragmentation, because a process partition is exactly the size of the process.

- Fig. Q.5.1 shows noncontiguous memory allocation method. Operating system maintain the table which contains the memory areas allocated to process and free memory. Memory management unit use this information for allocating processes.

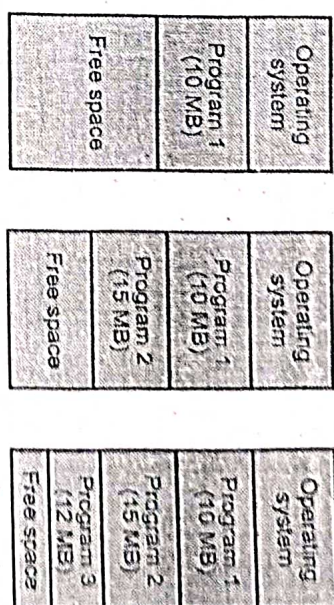


Fig. Q.5.1 (a) Variable size partition

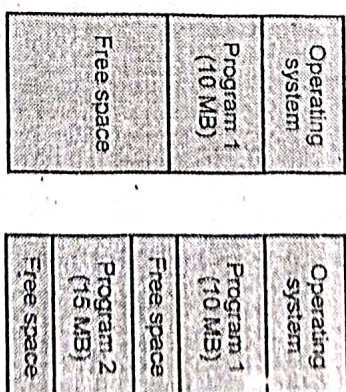


Fig. Q.5.1 (b) Noncontiguous memory allocation

- Table contains information about memory starting address for process/program and their size.
- CPU sends the logical address of the process to the MMU and the MMU uses the memory allocation information stored in the table for calculating logical address. We called this address as effective memory address of the data/instruction.



**Q.6 Explain difference between contiguous and non-contiguous memory allocation.**

**Ans. :** Difference between Contiguous and Noncontiguous Memory Allocation

| Sr. No. | Contiguous allocation                                 | Noncontiguous allocation                                |
|---------|-------------------------------------------------------|---------------------------------------------------------|
| 1.      | Program execution take place without overhead.        | Address translation is overhead.                        |
| 2.      | Swapped-in processes are placed in the original area. | Swapped-in processes can be placed any where in memory. |
| 3.      | Suffer from internal fragmentation.                   | Only paging, suffers from internal fragmentation.       |
| 4.      | Allocates single area of memory for process.          | Allocates more than one block of memory for process.    |
| 5.      | Wastage of memory.                                    | No wastage of memory.                                   |

**Q.7 What is fragmentation? Explain types of fragmentation.**

**Ans. :** • Fragmentation is a phenomenon in which storage space is used inefficiently, reducing capacity or performance and often both. The exact consequences of fragmentation depend on the specific system of storage allocation in use and the particular form of fragmentation.

• Fragmentation are of two types :

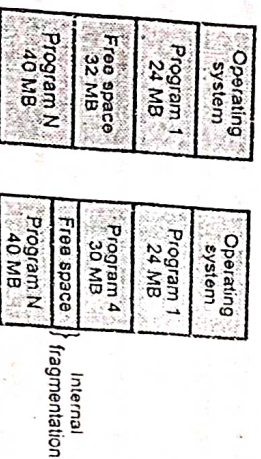
1. Internal fragmentation.
2. External fragmentation.

• In fragmentation, OS cannot use certain area of available memory.

**Internal fragmentation**

- There is wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition is called as internal fragmentation.

• Fig. Q.7.1 shows an internal fragmentation.



**Fig. Q.7.1 Internal fragmentation**

• To keep track of this free hole is overhead for system.

**External fragmentation**

- It occurs when enough total main memory space exists to satisfy a request, but it is not contiguous, storage is fragmented into a large number of small holes.

• Fig. Q.7.2 shows external fragmentation.

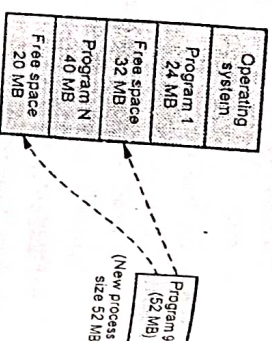
• Following are the solution for external fragmentation :

1. Compaction
2. Logical address space of a process/program

**Q.8 Explain the difference between internal and external fragmentation.**

**Ans. :**

| Sr. No. | Internal fragmentation                                                                                                                              | External fragmentation                                                                                                                    |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1.      | Memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is internal fragmentation. | External fragmentation exists when there is enough total memory space to satisfy a request, but available spaces are not contiguous.      |
| 2.      | First-fit and best-fit memory allocation does not suffer from internal fragmentation.                                                               | First-fit and best-fit memory allocation suffers from external fragmentation.                                                             |
| 3.      | Systems with fixed sized allocation units, such as the single partitions scheme and paging suffer from internal fragmentation.                      | Systems with variable sized allocation units, such as the multiple partitions scheme and segmentation suffer from external fragmentation. |



**Fig. Q.7.2 External fragmentation**



**Q.9 Explain different ways to remove external fragmentation.**

☞ [SPPU : May-18, Nov-19, End Sem, Marks 6]

- Operating system solves problem of external fragmentation.
- Operating system moves all the free holes to one side of main memory and creates large block of free size.
- It must be performed on each new allocation of process to memory or completion of process for memory. System must also maintain relocation information.
- All free blocks are brought together as one large block of free space. Compaction requires dynamic relocation.
- Fig. Q.9.1 shows compaction.

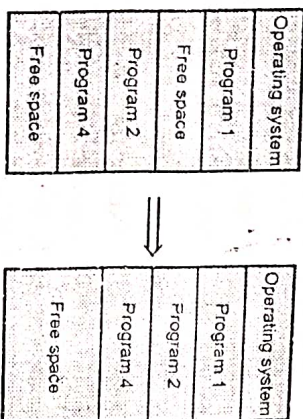


Fig. Q.9.1 Compaction

- Compaction has a cost and selection of an optimal compaction strategy is difficult. One method for compaction is swapping out those processes that are to be moved within the memory and swapping them into different memory locations.
  - Compaction is not always possible. Compaction is time consuming method and wastage of the CPU time.
- Garbage collection**
- Some programs use dynamic data structures. These programs dynamically use and discard memory space. Technically, the deleted data items release memory locations.

- But, in practice the OS does not collect such free space immediately for allocation. This is because that affects performance. Such areas, therefore are called garbage.
- When such garbage exceeds a certain threshold the OS would not have enough memory available for any further allocation.

**Q.10 Explain following terms in brief (any two) :**

- i) Internal fragmentation ii) External fragmentation iii) Compaction

☞ [SPPU : May-19, Dec-19 End Sem, Marks 4]

Ans. : i) Internal fragmentation : Refer Q.7.

ii) External fragmentation : Refer Q.7.

iii) Compaction : Refer Q.9.

**Q.11** Free memory holes of sizes 15 K, 10 K, 5 K, 25 K, 30 K, 40 K are available. The processes of size 12 K, 2 K, 25 K, 20 K, are to be allocated. How processes are placed using first fit, best fit and worst fit partitioning algorithm. Calculate internal and external fragmentation.

☞ [SPPU : May-18, Dec-18, 19, End Sem, Marks 10]

Ans. : Free memory holes = 15 K, 10 K, 5 K, 25 K, 30 K, 40 K

Process size = 12 K, 2 K, 25 K, 20 K

First Fit : 12 K → 15 K, 2 K → 10 K, 25 K → 25 K, 20 K → 30 K

Best Fit : 12 K → 15 K, 2 K → 5 K, 25 K → 25 K, 20 K → 30 K

Worst Fit : 12 K → 40 K, 2 K → 30 K, 25 K → 28 K (40 K - 12 K),

20 K → 28 K (30 K - 2 K)

**Q.12** Given memory partitions of 100 K, 500 K, 200 K, 300 K and 600 K (in-order), how would each of First-Fit, Best-Fit and Worst-Fit algorithms place processes of 212 K, 417 K, 112 K, 426 K? Which algorithm makes the most efficient use of memory?

☞ [SPPU : May-17, End sem, Marks 8]

Ans. : First-fit :

212 K is put in 500 K partition

417 K is put in 600 K partition

112 K is put in 288 K partition

(new partition 288 K = 500 K - 212 K)

426 K must wait.







- b) Allocates block of correct size
- c) It is easy to merge adjacent holes
- d) Fast to allocate memory and de-allocating memory

**Disadvantages**

- a) It requires all allocation unit to be powers of two
- b) It leads to internal fragmentation.
- c) Merging of holes is recursive, so poor worst-case behavior.

**4.4 : Paging****Q.14 What is paging ? Explain in details.**

Ans. : • In paging, operating system divides each incoming programs into pages of equal size. The sections of a disk are called block or sectors. The sections of main memory are called page frames. One sector will hold one page of job instructions and fit into one page frame of memory.

- In paging, logical address space of a program can be noncontiguous. It solves external fragmentation problem.
- The relation between virtual addresses and physical memory addresses given by page table.
- Fixed sized blocks are called frames and breaking of logical memory into blocks of same size called pages.
- Memory manager prepares following things before executing a program :
  1. Find out the number of pages in the program.
  2. Free space in the main memory.
  3. Loading of all the programs pages into memory.
- Pages are not loaded continuously in main memory. Each page can be stored in any available page frame anywhere in main memory. Memory manager keeps the track of pages of program. Paging avoids external fragmentation and need for compaction.

- Memory manager uses three tables to keep track of process and memory.

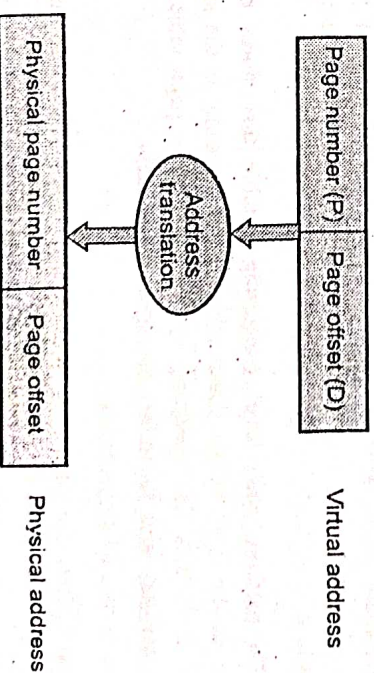
1. **Job table** : It stores the size of the active job and memory location where its page table is stored.
2. **Page map table** : It contains vital information about each page. PMT contains apage number and corresponding page frame memory address. It includes only one entry per page. Page numbers are in sequential order.
3. **Memory map table** : MMT is used to store page frame location and its status.

- Page size depends upon underlying hardware. Operating system maintains a page table for each process. The page table shows the frame location for each page of a process. Fig. Q.14.1 shows the virtual address format for paging system.

**Fig. Q.14.1 Virtual address format**

- Processor hardware performs the logical to physical address translation. Logical address contains page number and offset. The physical address contains frame number and offset. Offset is a relative factor. It is used to locate that line within its page frame.

- Fig. Q.14.2 shows address translation.

**Fig. Q.14.2 Address translation**



- Suppose a system uses  $n$ -bit for representing both physical and virtual address. The page number is represented by the most significant bit ( $n$  -  $m$  bits) and the displacement is represented by  $m$  bit.

- The table, which holds virtual address to physical address translations is called page table. As displacement is constant, so only translation of virtual page number to physical page is required.

**Q.15** What are the common techniques for structuring the page table? Explain at least three of these techniques.

[SPPU : Dec-17, End Sem, Marks 10]

**Ans. : Page Table Structure :**

1) Multilevel or hierarchical page table :

- Page tables can consume a significant amount of memory.

- For example : A 32-bit virtual address space using 4 KB pages.

$$\text{Page size} = 2^{12} \text{ bytes}$$

$$\text{Space for page numbers} = 2^{32} - 2^{12} = 2^{20} \text{ bytes}$$

- So page table may consists of upto 1 million entries, one for each page, for a total address capacity of about four billion bytes.

- Hierarchical page table is also called as forward mapped page table. This approach is so effective that many modern operating systems employ it.

- In this method, each level containing a table that stores pointers to tables in the level below. Each table in the hierarchy is the size of one page. It enables the operating system to transfer page tables between main memory and secondary storage device easily.

**For two levels of page table**

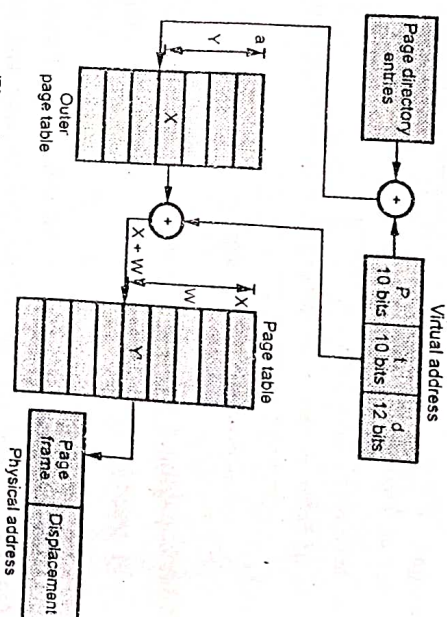
- Virtual address contains page number and displacement into that page.

$$\text{Virtual address (V)} = (p, t, d)$$

where  $(p, t) = \text{Page number}$

$$d = \text{Displacement}$$

- Here  $p$  is an index into the outer page table.
- Fig. Q.15.1 shows hierarchical page table.



**Fig. Q.15.1** Two level page table method

- Intel IA - 32 architecture support two levels of page tables.

**Advantages**

1. It is compact and support sparse address space.
2. It easily manage the memory.
3. It reduce table fragmentation.

**Disadvantages**

1. Overhead due to one more page table.
2. On TLB miss, two loads from memory will be required to get the right address translation information from the page table.

**2) Hashed page tables**

- Hashed page table support address space of 32 bits and more. Hash value is used as virtual page number.

- Hash table contains a link list of elements. Each elements consists of following fields :

1. Virtual page number
2. Mapped page frame value.
3. Pointer to the next element.

- Hash table improves search speed.

- Fig. Q.15.2 shows block diagram of hashed page table.



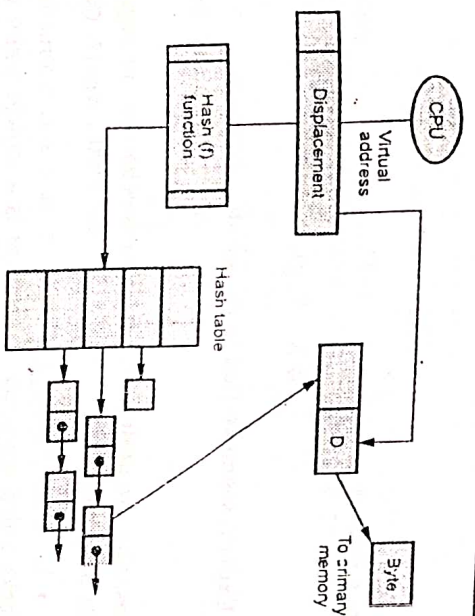


Fig. Q.15.2 Hashed page table

**Working**

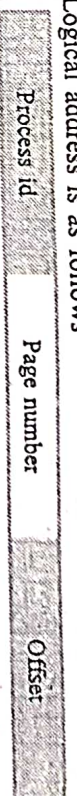
1. Virtual page number is taken from virtual address.
2. Virtual page number is hashed into the hash table.
3. Virtual page number is compared with the first element of linked list.
4. Both the value is matched, that value is (i.e. page frame) used for calculating physical address.
5. If the both value is not matched, the entire linked list is searched for a matching.

- Clustered page table is same as hashed page table but only difference is that each entry in the hash table refers to several pages.

**3) Inverted page table**

- It uses a page table that contains an entry for each physical frame. This ensure that the page table occupies a fixed fraction of memory. The size is proportional to the physical memory.
- Page table overhead increases with address space size. Page table get too big to fit in memory.
- Inverted page table has one entry for each real page of memory. Lookup time is increased because it requires a search on the inverted table.

- Logical address is as follows



- Inverted page table is used by Itanium, Power PC and Ultra SPARC.

**Q.16** Explain with the help of a neat diagram how TLB can be used to improve effective access time ? [5 PPU : Dec-17, End Sem, Marks 10]

- Ans. :**
- Set of registers are used for implementing page table. Registers are suitable only for small page table. If page table size increases then special purpose hardware cache is used for page table.
  - Special cache memory called a Translation Lookaside Buffer (TLB) is used with the address translation hardware. The full page table is kept in primary memory.

- TLB is very effective in improving the performance of page frame access. The cache buffer is implemented in a technology which is faster than the primary memory technology.

- Fig. Q.16.1 shows paging with TLB.

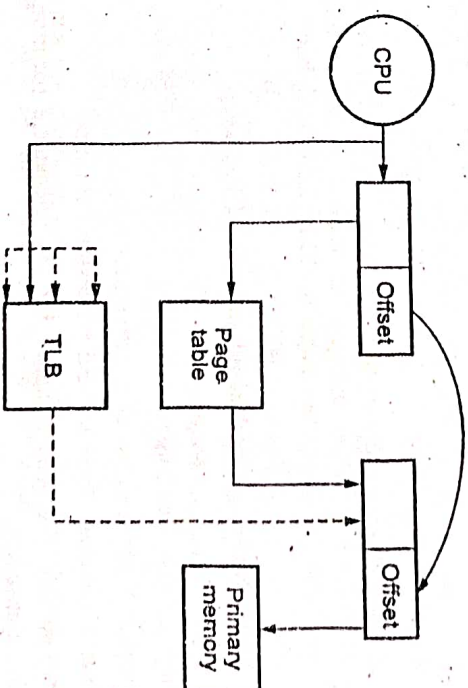


Fig. Q.16.1 Paging with TLB

- The TLB contents may be controlled by the operating system or by hardware, depending on the architecture.



- When a page is first translated to a page frame, the map is read from primary memory into the TLB. The TLB entry contains the page number, page frame's physical address etc.
- The page mapping mechanism first tries to find page number in the TLB. If the TLB contains the page number, then it is called as page hit or TLB hit.
- If the TLB does not contain an entry for page P, then it called as page miss or TLB miss. In case of TLB miss, the operating system locates the page table entry in the primary memory, which increases execution time.
- The TLB is associative, high-speed memory. A translation buffer is used to store a few of the translation table entries. It is very fast, but only for a small number of entries. On each memory reference.
  1. First ask TLB if it knows about the page. If so, the reference proceeds fast.
  2. If TLB has no information for page, must go through page and segment table to get information. Reference takes a long time, but gives the info for this page to TLB so it will know it for next reference.

#### Q.17 List advantages and disadvantages of Paging.

Ans. : **Advantages of Paging**

1. Paging eliminates fragmentation.
2. Support higher degree of multiprogramming.
3. Paging increases memory and processor utilization.
4. Compaction overhead required for the relocatable partition scheme is also eliminated.

**Disadvantages of Paging**

1. Page address mapping hardware usually increases the cost of the computer.
2. Memory must be used to store the various tables like page table, memory map table etc.
3. Some memory will still be unused if the number of available block is not sufficient for the address spaces of the jobs to be run.

#### 4.5 : Segmentation

#### Q.18 Explain segmentation with suitable diagram.

ISPPU : Dec.-18, End Sem, Marks

OR Describe the address translation mechanism in segmentation with suitable diagram.

ISPPU : May-19, End Sem, Marks

- Ans. : • In segmentation, a program's data and instructions are divided into blocks called segments. A segment is a logical entity in a program.
- Logical view : A process consists of a set of segments.
  - Physical view : It consists of non-adjacent areas of memory allocated segments.
  - All segment size may be equal or may not be equal. Segmentation support user view of memory.
  - Collection of segment is called as logical address space. Each segment is identified by its name.
  - Processor generates logical addresses. These addresses consist of segment number and an offset into the segment. Segment number is used as an index to segment page table.
  - Segment names are normally symbolic names.
  - Operating system maintains a segment table for each process. It is usually stored in main memory as a segment that is not to be loaded as long as the process can run.
  - Each entry in the segment table has a segment base and a segment limit. The base field contains the segment relocation register for the target segment. The segment limit field contains the length of the segment.
  - Fig. Q.18.1 shows an address translation in segmentation.
  - Segment table contains the physical address of the start of the segment. Then add the offset to the base and generate the physical address.



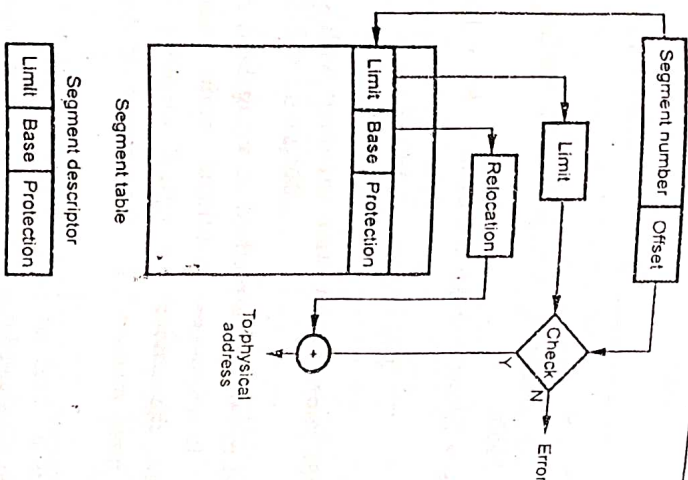


Fig. Q.18.1 Address translation in segmentation

- If the required reference is not found in one of the segment registers, then error is generated. At the same time, operating system does lookup in segment table and loads new segment descriptor into the register.

## Q.19 Compare segmentation and paging.

Ans. :

| Sr. No. | Segmentation                                                              | Paging                                                    |
|---------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| 1.      | Program is divided into variable size segments.                           | Program is divided into fixed size pages.                 |
| 2.      | User (or compiler) is responsible for dividing the program into segments. | Division into pages is performed by the operating system. |

|    |                                                                   |                                                                      |
|----|-------------------------------------------------------------------|----------------------------------------------------------------------|
| 3. | Segmentation is slower than paging.                               | Paging is faster than segmentation.                                  |
| 4. | Segmentation is visible to the user.                              | Paging is invisible to the user.                                     |
| 5. | Segmentation eliminates internal fragmentation.                   | Paging suffers from internal fragmentation.                          |
| 6. | Segmentation suffers from external fragmentation.                 | There is no external fragmentation.                                  |
| 7. | Processor uses page number, offset to calculate absolute address. | Processor uses segment number, offset to calculate absolute address. |
| 8. | OS maintain a list of free holes in main memory.                  | OS must maintain a free frame list.                                  |

Q.20 Explain the address translation mechanism in paging and segmentation with proper example. [SPPU : May-18, End Sem, Marks 8]

Ans. : Refer Q.14 and Q.18.

Q.21 Consider the following segment table :

| Segment | Base | Length |
|---------|------|--------|
| 0       | 219  | 600    |
| 1       | 2300 | 14     |
| 2       | 90   | 100    |
| 3       | 1327 | 580    |
| 4       | 1952 | 96     |

What are the physical addresses for the following logical addresses.

- i) 0, 430    ii) 1, 10    iii) 2, 80    iv) 3, 400    v) 4, 90

[SPPU : May-16, End sem, Marks 8]

Ans. : Physical Address = Offset + Segment Base

- i) 0, 430

Offset = 430



Segment length = 600  
Segment = 0

Offset 430 is less than segment length 600 and starting location of segment 0 is start from 219.

Physical address = Offset + Segment Base =  $430 + 219 = 649$

ii) 1, 10

Offset = 10

Segment length = 14

Segment = 0

Offset 10 is less than segment length 14 and starting location of segment 1 is start from 2300.

Physical address = Offset + Segment Base =  $10 + 2300 = 2310$

iii) 2, 80

Offset = 80

Segment length = 100

Segment = 2

Offset 80 is less than segment length 100 and starting location of segment 2 is start from 90.

Physical address = Offset + Segment Base =  $80 + 90$

Physical address = 170

iv) 3, 400

Offset = 400

Segment length = 580

Segment = 3

Offset 400 is less than segment length 580 and starting location of segment 3 is start from 1327.

Physical address = Offset + Segment Base =  $400 + 1327 = 1727$

v) 4, 90

Offset = 90

Segment length = 96

RECODE

Segment = 4

Offset 90 is less than segment length 96 and starting location of segment 4 is start from 1952.

Physical address = Offset + Segment Base =  $90 + 1952 = 2042$

#### 4.6 : Virtual Memory

Q.22 Explain the concept of virtual memory with suitable diagram.

Ans. [SPPU : Dec-18, End Sem, Marks 8]

Ans. : • Virtual memory is a method of using hard disk space to provide extra memory. It simulates additional main memory. In Windows operating system, the amount of virtual memory available equals the amount of free main memory plus the amount of disk space allocated to the swap file.

- A swap file is an area of your hard disk that is set aside for virtual memory. Swap files can be either temporary or permanent.
- Virtual memory is stored in the secondary storage device. It helps to extend additional memory capacity and work with primary memory to load applications. The virtual memory will reduce the cost of expanding the capacity of physical memory.
- Each process address space is partitioned into parts that can be loaded into primary memory when they are needed and written back to secondary storage otherwise. Address space partitions have been used for the code, data and stack identified by the compiler and relocating hardware.
- The portion of the process that is actually in main memory at any time is defined to be the resident set of the process. The logical addressable space is referred to as virtual memory.
- The virtual address space is much larger than the physical primary memory in a computer system. The virtual memory works with the help of secondary storage device and its speed is low compared to the physical storage location.

RECODE



- Virtual memory uses two types of addresses : Virtual address and physical address

• Fig. Q.22.1 shows concept of virtual memory.

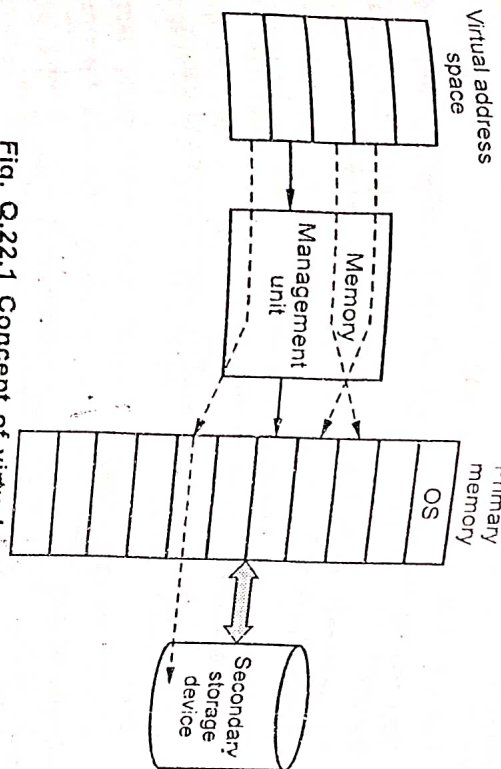


Fig. Q.22.1 Concept of virtual memory .

- Virtual address is referred by process and physical address is actual address of the main memory.
- Whenever a process accesses a virtual address, the system must translate it to a physical address. Virtual memory system uses special purpose hardware. This hardware is called as memory management unit.
- Most virtual memory system use a technique called paging. The virtual address space is divided into fixed size units called pages.
- The corresponding units in the physical memory are called page frames. The pages and page frames are normally the same size. The area on a hard disk that stores page frames is usually called the paging file or the swap file.
- When the system is ready to run a process, the system loads the process code and data from secondary storage into primary memory. Only a small portion of these needs to be in primary memory at once for the process to execute. The success of implementing virtual memory system

is how to map virtual addresses to physical addresses. Because processes reference only virtual addresses.

#### 4.7 : Demand Paging

**Q.23** Elaborate the concept of demand paging with appropriate diagram.

Ans : • In demand paging, a page is loaded into primary memory only when the process references it. It is the simplest fetch policy implemented in virtual memory.

- Demand paging guarantees that the system brings into primary memory only those pages that processes actually need. It involves interaction between hardware and software component of the virtual memory.
- Paging with swapping method will become as demand paging concept. Process's logical address space is stored on the secondary storage device.
- Demand paging requires that the page map table for each process keep track of each page as it is loaded or removed from primary memory.
- Fig. Q.23.1 shows the concept of demand paging.

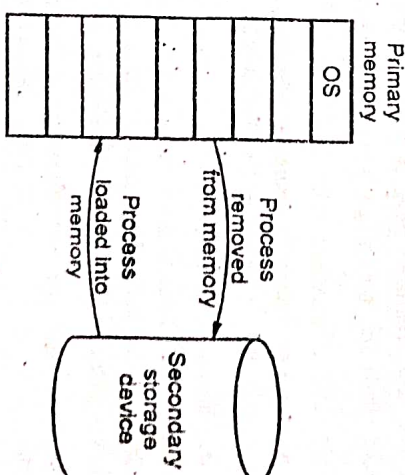


Fig. Q.23.1 Demand paging



- Working of demand paging

1. Program attempts to find a page.
2. If the page is located in the primary memory, then the program executes.
3. If the page cannot be found in the primary memory, then the page fault occurs.
4. Memory reference is checked for required page to determine it is a valid reference to a location on a secondary storage, if it is then the page will be required page. If not, the process is terminated.
5. Schedule disk operation to read the required page into primary memory.
6. Restart the instruction that was interrupted by the OS trap.

- Demand paging takes support from hardware to distinguish the pages. The valid-invalid bit method is used. This method gives the idea about page location. Page may be in the primary memory or in the secondary storage.

- If valid bit is set, then required page is stored in the primary memory. When an invalid bit is set, then page is on the secondary storage disk.

- Valid-invalid bit field is attached with page table. The valid-invalid bit field contain a Boolean value to indicate whether the page exists in primary memory. 0 = Not in memory 1 = In memory

- Initially all valid-invalid entries are set to 0. This information is used for page replacement algorithm. Page fault leads to the page replacement operation to load the required pages in primary memory. A failure to find a page in memory is often called a page fault.

**Q.24** What are the steps in handling page fault? Explain with suitable diagram.

[SPPU : May-17, End Sem, Marks 10]

**Ans. : Steps in Handling a Page Fault**

- When page fault occurs, the system performs following steps :

1. Check the process control block table for reference which bit is set i.e. valid bit or invalid bit for memory access.
2. If invalid bit is set, user will terminate the process. If valid bit is set, then the page is loaded into memory.

3. Check for free frame for new page loading.
4. Start reading secondary storage device for desired page for allocation frame.
5. Required page is loaded into the memory after completion of operation. PCB table is updated.
6. System restart the instruction that was interrupted by the error. Page loaded in memory so process can use it.

- Fig. Q.24.1 shows page fault handling steps in demand paging.

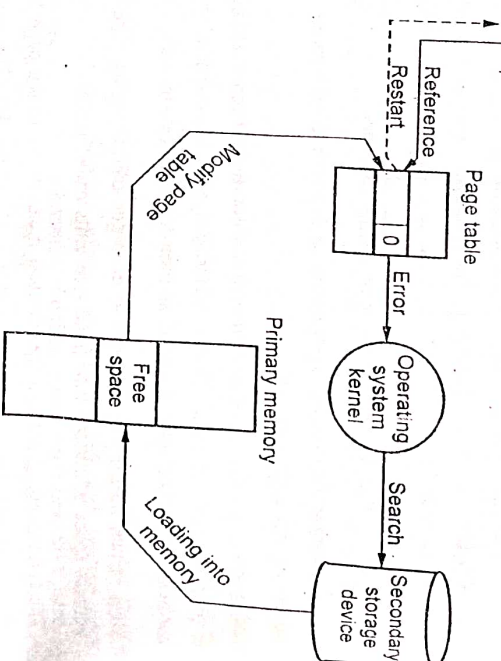


Fig. Q.24.1 Handling of page fault

#### 4.8 : Page Replacement

**Q.25** What is page replacement ? List the name of page replacement algorithm.

**Ans. : •** Whenever a process requests data from a page which is not in the memory then the system uses a page replacement algorithm to swap out an existing page to make space for the new page.

- Page replacement algorithm is used to decide which page will be replaced to allocate memory to the current referenced page.



- There are three page replacement algorithms :

1. First In First Out (FIFO)
2. Least Recently Used First (LRU)
3. Optimal Page Replacement

- In FIFO algorithm the system keeps track of the time when each page gets into the memory. For page replacement, the page which entered first into the memory is selected for replacement. This technique suffers from Belady's Anomaly.

- LRU page replacement algorithm associates with each page the time of that page's last use. When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.

- Optimal page replacement looks in future requests. Optimal page replacement algorithm replaces the page that will not be used for the longest period of time. Unfortunately, there is no way to determine which page will be last, so in practice this algorithm cannot be use.

Q.26 For the following reference string.

1,2,3,4,2,1,5,6,2,1,2,3,3,6. Count the number of page faults that occur with 3 frames using FIFO and LRU page replacement methods. Discuss the result.

[SPPU : De.c-17, End Sem, Marks 8]

Ans. : FIFO :

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 1 | 2 | 3 | 4 | 2 | 1 | 5 | 6 | 2 | 1 | 2 | 3 | 3 | 6 |
| 1          | 1 | 1 | 1 | 4 | 4 | 4 | 4 | 6 | 6 | 6 | 6 | 3 | 3 | 3 |
| 2          | - | 2 | 2 | 2 | 2 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 6 |
| 3          | - | - | 3 | 3 | 3 | 3 | 5 | 5 | 5 | 1 | 1 | 1 | 1 | 1 |
| Page Fault |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

Total number of page fault = 11

LRU :

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 1 | 2 | 3 | 4 | 2 | 1 | 5 | 6 | 2 | 1 | 2 | 3 | 3 | 6 |
| 1          | 1 | 1 | 1 | 4 | 4 | 4 | 5 | 5 | 5 | 1 | 1 | 1 | 1 | 6 |
| 2          |   | 2 | 2 | 2 | 2 | 2 | 2 | 6 | 6 | 6 | 6 | 3 | 3 | 3 |
| 3          |   |   | 3 | 3 | 3 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 |
| Page Fault |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

Total number of page fault = 11

Q.27 For the following reference string.

5,6,7,8,5,6,9,5,6,7,8,9,6,7,4,9,8

Count the number of page fault occur with 3 frames using FIFO, optimal and LRU page replacement methods. Discuss the result.

[SPPU : May-18, End sem, Marks 12]

Ans. : FIFO :

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 5 | 6 | 7 | 8 | 5 | 6 | 9 | 5 | 6 | 7 | 8 | 9 | 6 | 7 | 4 | 9 | 8 |
| 1          | 5 | 5 | 5 | 8 | 8 | 8 | 9 | 9 | 9 | 9 | 9 | 9 | 6 | 6 | 6 | 6 | 8 |
| 2          | - | 6 | 6 | 6 | 5 | 5 | 5 | 5 | 5 | 7 | 7 | 7 | 7 | 7 | 4 | 4 | 4 |
| 3          | - | - | 7 | 7 | 7 | 7 | 6 | 6 | 6 | 6 | 8 | 8 | 8 | 8 | 8 | 9 | 9 |
| Page Fault |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

Total number of page fault = 13

LRU :

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 5 | 6 | 7 | 8 | 5 | 6 | 9 | 5 | 6 | 7 | 8 | 9 | 6 | 7 | 4 | 9 | 8 |
| 1          | 5 | 5 | 5 | 8 | 8 | 8 | 9 | 9 | 9 | 7 | 7 | 6 | 6 | 6 | 9 | 9 | 9 |
| 2          | - | 6 | 6 | 6 | 5 | 5 | 5 | 5 | 5 | 8 | 8 | 8 | 8 | 7 | 7 | 7 | 8 |
| 3          | - | - | 7 | 7 | 7 | 7 | 6 | 6 | 6 | 6 | 6 | 9 | 9 | 9 | 4 | 4 | 4 |
| Page Fault |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

Total number of page fault = 15



Optimal :

|            |    |    |    |    |   |   |    |   |   |    |    |   |   |   |    |    |   |
|------------|----|----|----|----|---|---|----|---|---|----|----|---|---|---|----|----|---|
|            | 5  | 6  | 7  | 8  | 5 | 6 | 9  | 5 | 6 | 7  | 8  | 9 | 6 | 7 | 4  | 9  | 8 |
| 1          | 5  | 5  | 5  | 5  | 5 | 5 | 5  | 5 | 5 | 7  | 8  | 8 | 8 | 8 | 8  | 8  | 8 |
| 2          | -  | 6  | 6  | 6  | 6 | 6 | 6  | 6 | 6 | 6  | 6  | 6 | 6 | 6 | 7  | 4  | 4 |
| 3          | -  | -  | 7  | 8  | 8 | 9 | 9  | 9 | 9 | 9  | 9  | 9 | 9 | 9 | 9  | 9  | 9 |
| Page Fault | PF | PF | PF | PF |   |   | PF |   |   | PF | PF |   |   |   | PF | PF |   |

Total number of page fault = 9

Q.28 For the following reference string, 6, 5, 1, 2, 5, 3, 5, 4, 2, 3, 6, 3, 2, 1, 2 Count the number of page faults that occur with 3 frames using FIFO, Optimal and LRU page replacement methods. Discuss the result.

Ans. : FIFO (Page frame = 3)

[SPPU : Dec-18, 19, End Sem, Marks 12]

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 6 | 5 | 1 | 2 | 5 | 3 | 5 | 4 | 2 | 3 | 6 | 3 | 2 | 1 | 2 |
| 1          | 6 | 6 | 6 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 6 | 6 | 6 | 1 | 1 |
| 2          | - | 5 | 5 | 5 | 5 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 3          | - | - | 1 | 1 | 1 | 1 | 5 | 5 | 5 | 3 | 3 | 3 | 3 | 3 | 3 |
| Page Fault | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P |
| Fault      | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F |

Total number of page fault = 12

Optimal (Page frame = 3)

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 6 | 5 | 1 | 2 | 5 | 3 | 5 | 4 | 2 | 3 | 6 | 3 | 2 | 1 | 2 |
| 1          | 6 | 6 | 6 | 6 | 6 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 1 | 1 |
| 2          | - | 5 | 5 | 5 | 5 | 5 | 4 | 4 | 4 | 4 | 6 | 6 | 6 | 6 | 6 |
| 3          | - | - | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Page Fault | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P |
| Fault      | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F |

Total number of page fault = 8

LRU (page frame = 3)

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 6 | 5 | 1 | 2 | 5 | 3 | 5 | 4 | 2 | 3 | 6 | 3 | 2 | 1 | 2 |
| 1          | 6 | 6 | 6 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 6 | 6 | 6 | 1 | 1 |
| 2          | - | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 3 | 3 | 3 | 3 | 3 | 3 |
| 3          | - | - | 1 | 1 | 1 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| Page Fault | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P |
| Fault      | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F |

Total number of page fault = 9

Q.29 For the following reference string 0, 1, 3, 6, 2, 4, 5, 2, 5, 0, 3, 1, 2, 5, 4, 1, 0 count the number of page faults that occur with 3 frames using FIFO, Optimal and LRU page replacement methods. Discuss the results.

Ans. : FIFO (page frame = 3)

[SPPU : May-19, End Sem, Marks 12]

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 0 | 1 | 3 | 6 | 2 | 4 | 5 | 2 | 5 | 0 | 3 | 1 | 2 | 5 | 4 | 1 | 0 |
| 1          | 0 | 0 | 0 | 6 | 6 | 6 | 5 | 5 | 5 | 5 | 5 | 1 | 1 | 1 | 4 | 4 | 4 |
| 2          | - | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 0 | 0 | 0 | 2 | 2 | 2 | 1 | 1 |
| 3          | - | - | 3 | 3 | 3 | 3 | 4 | 4 | 4 | 4 | 3 | 3 | 3 | 5 | 5 | 5 | 0 |
| Page Fault | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P |
| Fault      | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F |

Total number of page fault = 15

Optimal (Page frame = 3)

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 0 | 1 | 3 | 6 | 2 | 4 | 5 | 2 | 5 | 0 | 3 | 1 | 2 | 5 | 4 | 1 | 0 |
| 1          | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2          | - | 1 | 1 | 6 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 4 | 4 | 4 |
| 3          | - | - | 3 | 3 | 3 | 3 | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 0 |
| Page Fault | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P | P |
| Fault      | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F | F |



Total number of page fault = 11

LRU (Page frame = 3)

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0          | 1  | 3  | 6  | 2  | 4  | 5  | 2  | 5  | 0  | 3  | 1  | 2  | 5  | 4  | 1  | 0  |
| 0          | 0  | 0  | 6  | 6  | 6  | 5  | 5  | 5  | 5  | 5  | 1  | 1  | 1  | 4  | 4  | 4  |
| 1          | 1  | 1  | 1  | 2  | 2  | 2  | 2  | 2  | 2  | 3  | 3  | 3  | 5  | 5  | 5  | 0  |
| 2          | -  | 3  | 3  | 3  | 4  | 4  | 4  | 4  | 4  | 0  | 0  | 2  | 2  | 2  | 1  | 1  |
| 3          | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| Page fault | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF | PF |

Total number of page fault = 15

#### 4.9 : Allocation of Frames, Thrashing

Q.30 Explain thrashing.

Ans. : • Thrashing occurs when a process does not have "enough" frames allocated to store the pages it uses repeatedly, the page fault rate will be very high.

- A process is thrashing if it is spending more time for paging in/out than executing. Since thrashing leads to low CPU utilization, the operating system's medium or long term scheduler may step in, detect this and increase the degree of multiprogramming.
- Local replacement algorithms can limit the effects of thrashing. If the degree of multiprogramming is increased over a limit, processor utilization falls down considerably because of thrashing. Fig. Q.30.1 shows thrashing.

• The selection of a replacement policy to implement virtual memory plays an important part in the elimination of the potential for thrashing. A policy based on the local mode will tend to limit the effect of thrashing. Replacement policy based on the global mode is more likely to cause thrashing. Since all pages of memory are available to all transactions, a memory-intensive transaction may occupy a large portion of memory, making other transactions susceptible to page faults and resulting in a system that thrashes.

- Thrashing is solved by using working set model and page fault frequency.

Q.31 Write short note on locality of reference.

Ans. : • Locality of reference is also known as the principle of locality. Many applications continually reference large amounts of data.

- Locality of reference is observed that an application does not access all of its data at once with equal probability. Instead, it accesses only a small portion of it at any given time.

• There are two basic types of reference locality.

1. Temporal locality
  2. Spatial locality
1. Temporal locality : Temporal locality is locality over time. If at one point in time a particular memory location is referenced, then it is likely that the same location will be referenced again in the near future.
2. Spatial locality : If a particular memory location is referenced at a particular time, then it is likely that nearby memory locations will be referenced in the near future.

END...S

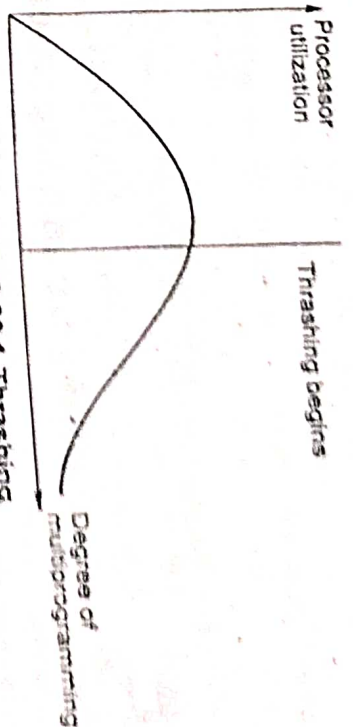


Fig. Q.30.1 Thrashing